

BY: Ewan (2slashes//), John (KungPaoMeow), Junyi (Martin-SSS).

Flask app - Medicine Management System for a Pharmacy

BY: Ewan (2slashes//), John (KungPaoMeow), Junyi (Martin-SSS).

This project is a Flask-based web application that manages drug information and order tracking. It allows users to add, edit, delete, and browse drugs as well as manage customer orders. The project also includes a suite of unit tests to ensure its functionality.

Requirements

Before you can run the project, ensure you have the following dependencies installed:

Python 3.7+

Flask

Unittest (Python's built-in module)

datetime

Running the Application

To start the Flask application, execute the following command in the project directory:

```
python app.py
```

By default, the application will run on <http://127.0.0.1:5001/>. You can access the following routes:

- **Dashboard:** <http://127.0.0.1:5001/dashboard>
- **Browse Drugs:** <http://127.0.0.1:5001/browse-drug>
- **Order Tracking:** <http://127.0.0.1:5001/order-tracking>

Project Structure

The key files in this project include:

- `app.py`: Main application logic and route handlers.

BY: Ewan (2slashes//), John (KungPaoMeow), Junyi (Martin-SSS).

- `unit_tests.py`: Unit test scripts for testing drug and order management functionalities.
- `templates/`: HTML templates for rendering web pages.
- `static/`: Static files such as CSS or JavaScript (if used).

Running Unit Tests

The unit tests are located in `unit_tests.py` and use the `unittest` framework. The tests cover adding, editing, deleting drugs and orders, as well as checking search and edge cases.

To execute the unit tests, you can run the following command in the project directory:

```
python -m unittest unit_tests.py
```

Example Tests Covered

- Adding new drugs and orders.
- Editing existing drugs and orders.
- Deleting drugs and orders.
- Searching for drugs.
- Testing edge cases such as invalid inputs or missing data.